

Tabular Reinforcement Learning for Text Flappy Bird: A Comparative Study of MC and Sarsa(λ)

Aurèle GALLARD

CentraleSupélec

1 Introduction and Experimental Setup

This study compares tabular Reinforcement Learning (RL) agents on the *Text Flappy Bird* environment. The provided `screen-v0` environment yields a full 2D grid, rendering tabular RL computationally unfeasible due to the *Curse of Dimensionality*. Conversely, the `v0` environment yields only relative distances (dx, dy) , keeping the state space compact but myopic (lacking velocity and awareness of upcoming obstacles).

To achieve an optimal balance, we implemented a custom `ObservationWrapper` to extract a 4-dimensional state from the raw screen: $S = (dx_1, dy_1, v_y, dy_2)$, representing the horizontal and vertical distances to the first pipe, the bird's vertical velocity, and the vertical distance to the second pipe.

2 Implemented Agents

Two tabular control algorithms were implemented and trained for 50,000 episodes using an ϵ -greedy policy ($\epsilon = 0.1$):

1. **Monte Carlo First-Visit:** Updates Q-values at the end of each episode based on the total return. It is unbiased but suffers from high variance.
2. **Sarsa(λ):** A Temporal Difference (TD) algorithm that updates step-by-step. We implemented *replacing eligibility traces* (with threshold pruning for optimization) to propagate rewards backward. We used a learning rate $\alpha = 0.1$ and a discount factor $\gamma = 1.0$.

3 Results and Performance Comparison

3.1 Learning Curves and Parameter Sweep

Figure 1 demonstrates the comparative performance of the agents. Initially, Monte Carlo exhibits faster convergence than Sarsa(λ). Because MC updates values using the full episode return, early "lucky" trajectories rapidly distribute high values across the state space. Conversely, Sarsa's step-by-step Temporal Difference updates require more time to propagate through our large 4D state space. However, as training progresses, Sarsa's lower variance allows it to stabilize and ultimately surpass Monte Carlo. A parameter sweep of the trace-decay parameter $\lambda \in \{0.0, 0.5, 0.9\}$ confirms that $\lambda = 0.9$ yields the fastest learning, as it effectively propagates collision penalties backward along the trajectory.

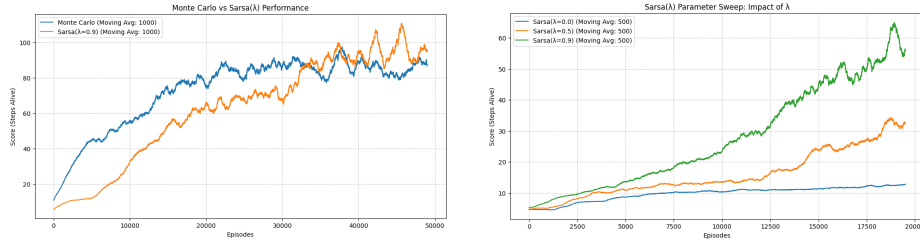


Fig. 1. Smoothed learning curves for both agents (left) and Sarsa(λ) parameter sweep (right).

3.2 State-Value Function Visualization

Figure 2 visualizes a 2D slice of the learned State-Value function $V(s) = \max_a Q(s, a)$ for both agents, fixing $v_y = -1$ (the moment after a "flap" action) and $dy_2 = \text{None}$.

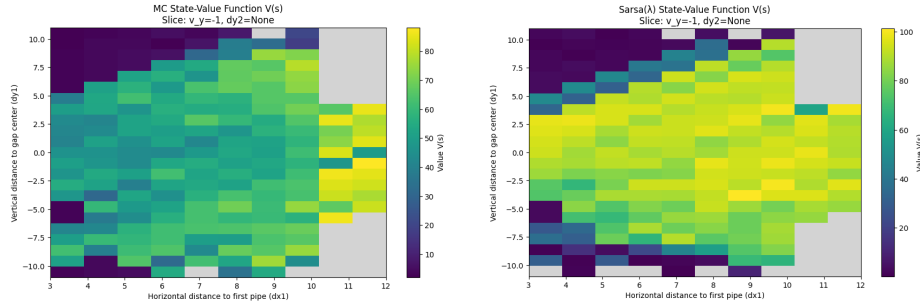


Fig. 2. State-Value functions $V(s)$ for Monte Carlo (left) and Sarsa(λ) (right). $dx_1 \in \{0, 1, 2\}$ is not covered because we cut the space at $dy_2 = \text{None}$ and for low dx_1 the second gate is already visible on the screen.

The resulting maps are highly interpretable. We observe a distinct "yellow arrow" (or funnel) pointing left towards $dx_1 = 0$. This indicates that as the bird approaches the pipe horizontally, the safe vertical tolerance dy_1 narrows significantly. The dark blue triangles at the top and bottom left represent highly negative expected returns, corresponding to states where a crash into the pipes is mathematically unavoidable. Furthermore, the grey areas at the top right and bottom right represent unreachable states; because the bird spawns in the center of the screen, it is physically impossible to reach extreme vertical distances when the horizontal distance dx_1 is still at its maximum starting value. The Sarsa(λ) map exhibits much cleaner and sharper contours than the Monte Carlo map, reflecting its lower variance.

4 Robustness and Generalization

To test generalization, the trained Sarsa(λ) agent was evaluated without further training on modified level configurations (Figure 3).

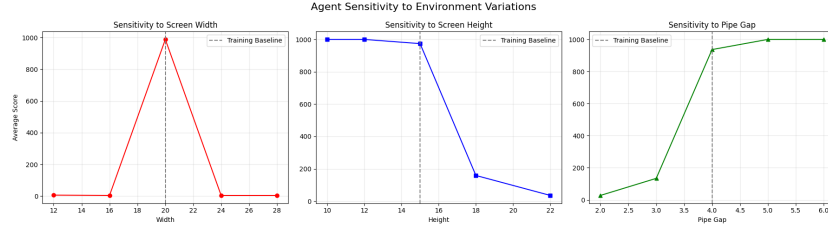


Fig. 3. Sensitivity analysis: Agent performance under data distribution shifts.

Height: The agent is robust to decreases in screen height because its logic relies on the relative vertical distance dy_1 (translation invariance). However, drastically increasing the height introduces massive, previously unvisited vertical distances, leading to a performance drop.

Width: Modifying the screen width fundamentally breaks our custom agent. A wider screen shifts the visual horizon, causing the second pipe (dy_2) to become visible at dx_1 distances where it was previously **None** during training. The agent encounters entirely unknown states (*Data Distribution Shift*) and fails. Ironically, the simple **TextFlappyBird-v0** environment (which ignores dy_2) would be perfectly robust to width modifications.

Pipe Gap: Increasing the gap eases the physical constraints, allowing the agent to survive indefinitely. Shrinking the gap removes the margin of error learned during training, requiring pixel-perfect actions the agent has not explored, resulting in immediate failure.

5 Transitioning to the Original Environment

The implemented tabular agents cannot be directly used on the original **flappy-bird-gym** environment. The original game operates on a continuous physics engine with floating-point coordinates and velocities. Applying tabular algorithms here would yield an infinite state space, preventing Q-table convergence. Solving the original environment requires either severe state discretization (binning, which loses critical precision) or transitioning to Deep Reinforcement Learning (e.g., DQN, PPO) for continuous function approximation.

6 Conclusion

This study highlights the faster convergence of Sarsa(λ) over Monte Carlo. Crucially, it demonstrates that while complex state representations (like our 4D custom state) yield highly performant agents, they remain highly susceptible to data distribution shifts compared to simpler, translation-invariant abstractions.

7 Code Availability

The notebook containing the implementations and visualizations is publicly available at <https://github.com/AureleGallard/RL-Text-Flappy-Bird.git>